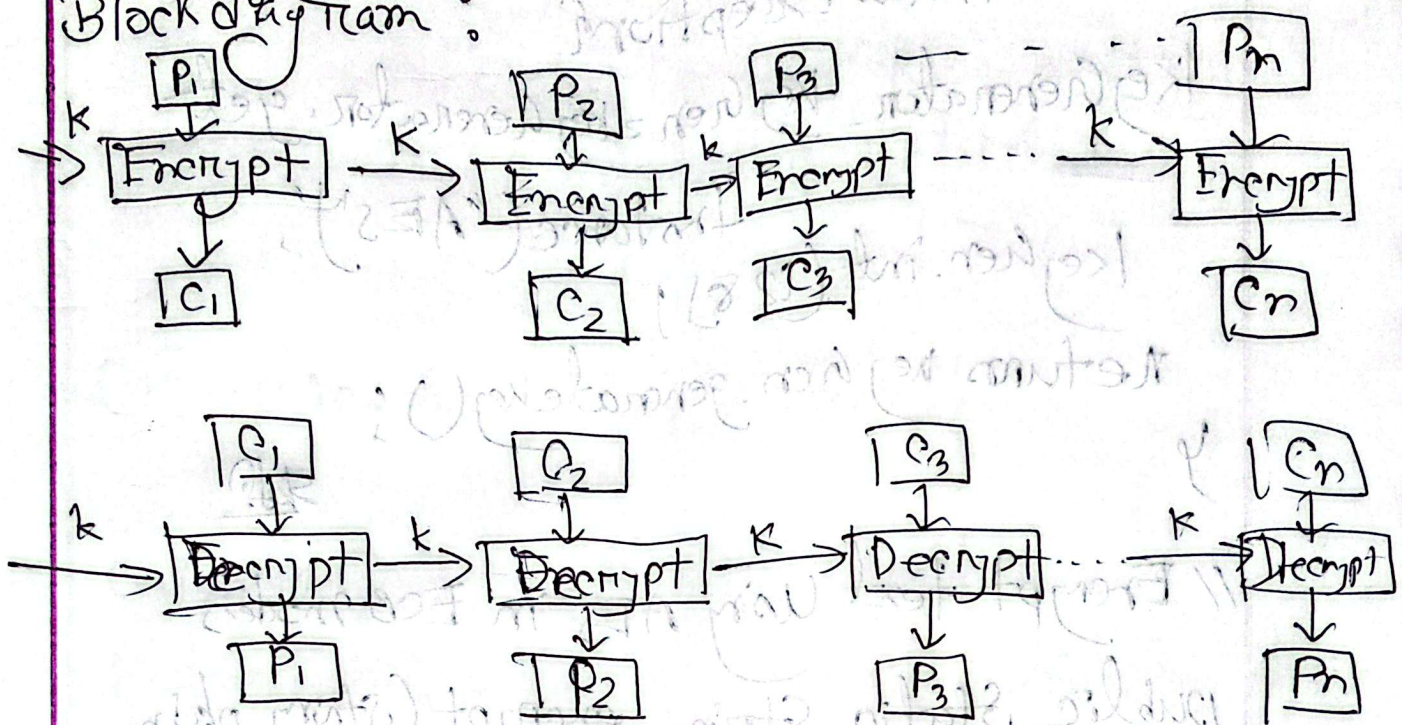


Assignment

IT-21010

ECB (Electronic codebook): In an electronic code book, each block of bits of plaintext is encoded independently with the same key.

Block diagram:



Java Implementation

```
import javax.crypto.Cipher;  
import javax.crypto.KeyGenerator;  
import javax.crypto.SecretKey;  
import javax.crypto.spec.SecretKeySpec;  
import java.util.Base64;
```

IT-21010

```
public class ECB mode Example {  
    // generate a sample AES key (128 bit)  
    public static SecretKey generateAESKey()  
        throws Exception {
```

```
        KeyGenerator keyGen = KeyGenerator.getInstance("AES");  
        keyGen.init(128);  
        return keyGen.generateKey();  
    }  
}
```

```
// Encrypt text using AES in ECB mode  
public static String encrypt(String plainText, SecretKey key) throws Exception {  
    Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5padding");  
    cipher.init(Cipher.ENCRYPT_MODE, key);  
    byte[] encryptBytes = cipher.doFinal(plainText.getBytes());  
    return Base64.getEncoder().encodeToString(encryptBytes);  
}
```

```

// Decrypt cipher text using AES in ECB mode
public static String decrypt (String encrypted Text,
    Secret Key) throws Exception {
    Cipher cipher = cipher.getInstance ("AES/ECB/PKCS5
        padding");
    cipher.init (Cipher.DECRYPT_MODE, Secret Key);
    byte [] decrypted Bytes = cipher.doFinal (Base 64.
        getDecoder (). decode (encrypted Text));
    return new String (decrypted Bytes);
}

```

Output:

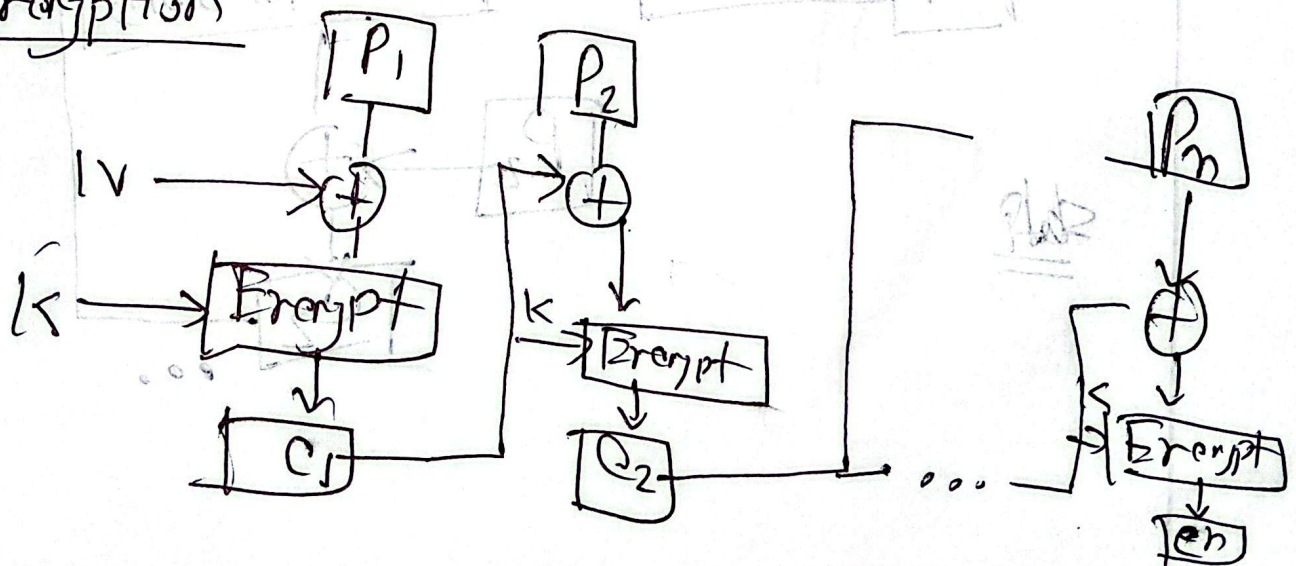
Original Text: Hello cyber world;

Encrypted Text: fRQ+HXKBP3A8CXIV9IGWF19VHk

Decrypted Text: Hello cyber world;

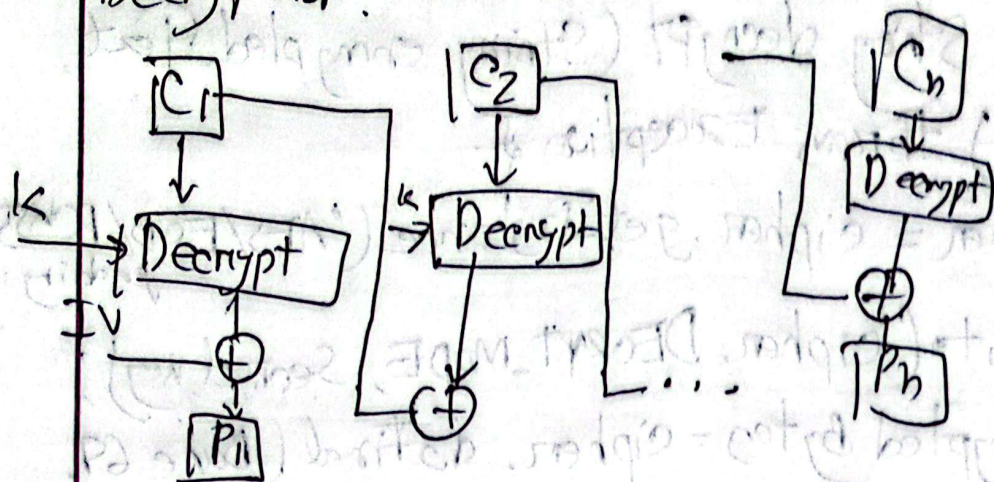
CBC (Cipher Block Chaining)

Encryption



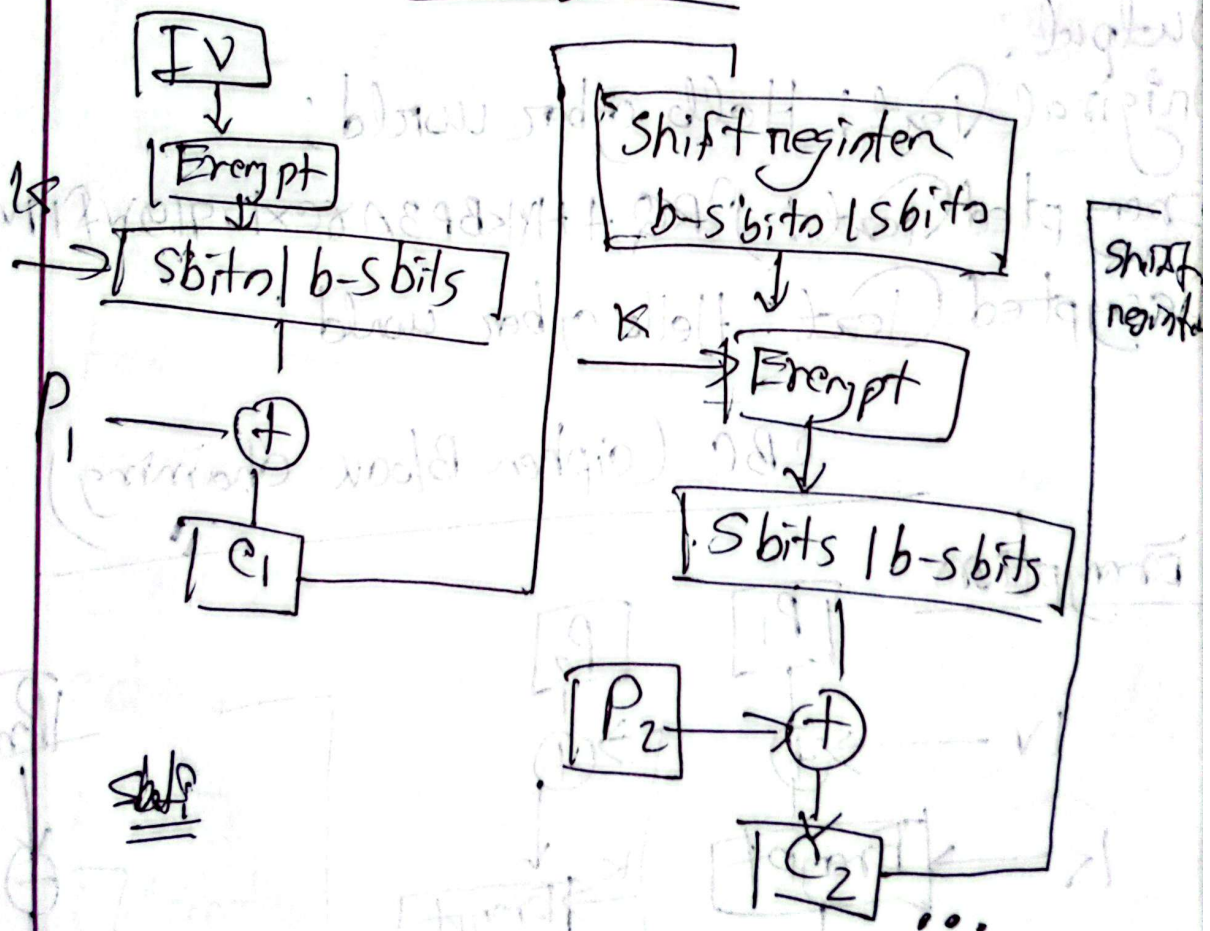
IT-21010

Decryption:



Cipher Feedback Mode (CFB)

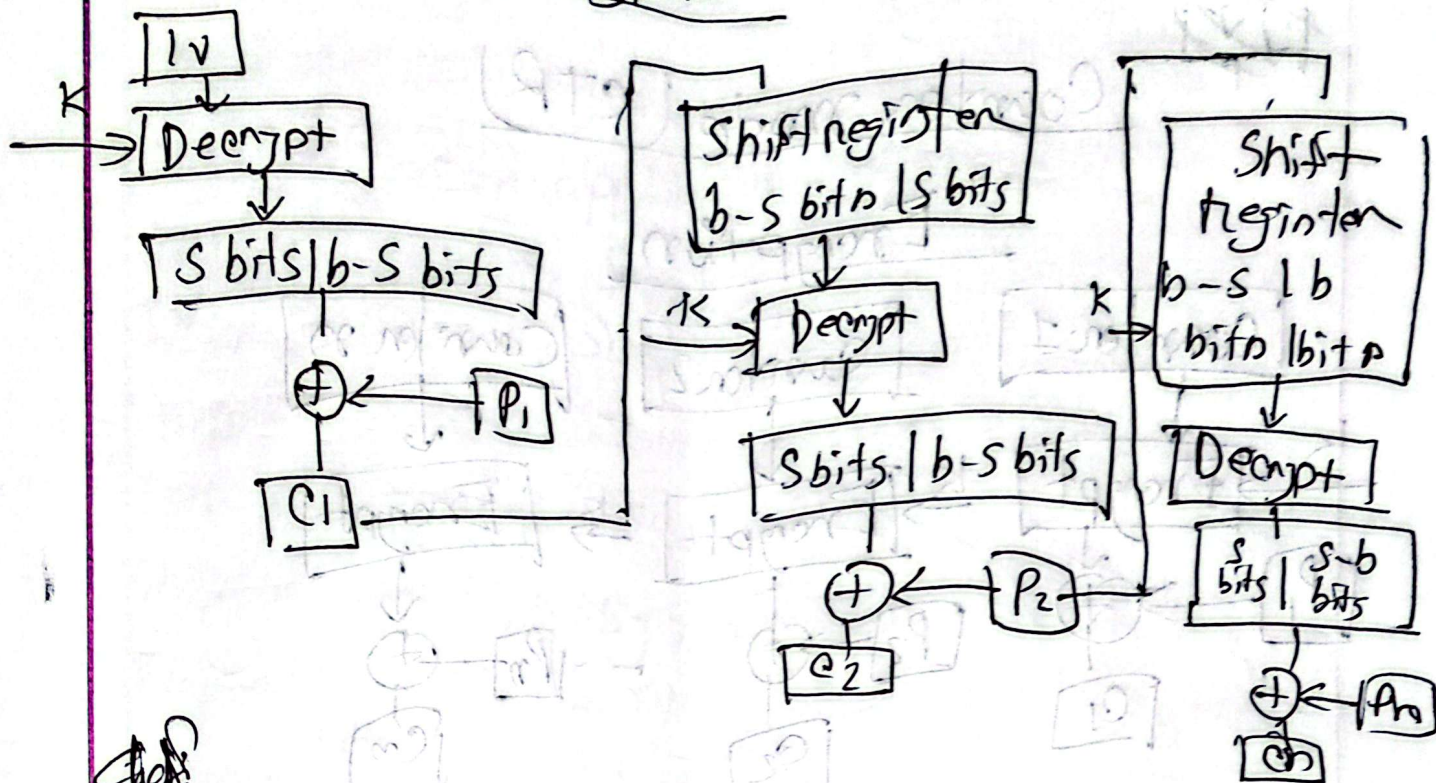
Encryption



IT-21010

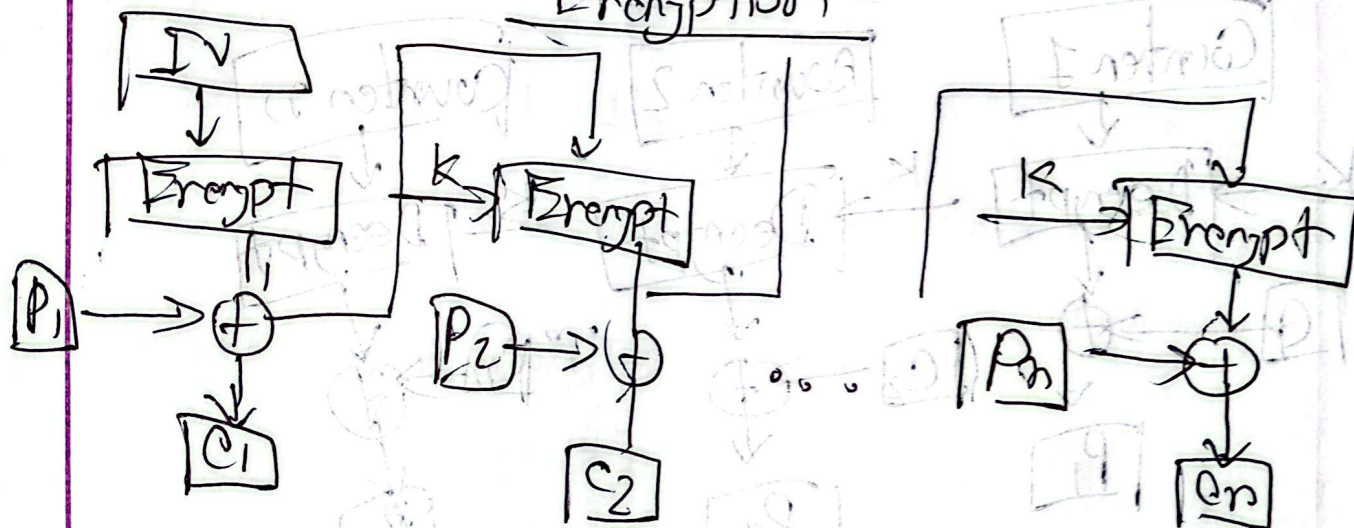
0105-TI

Decryption



Output Feedback (OFB)

Encryption



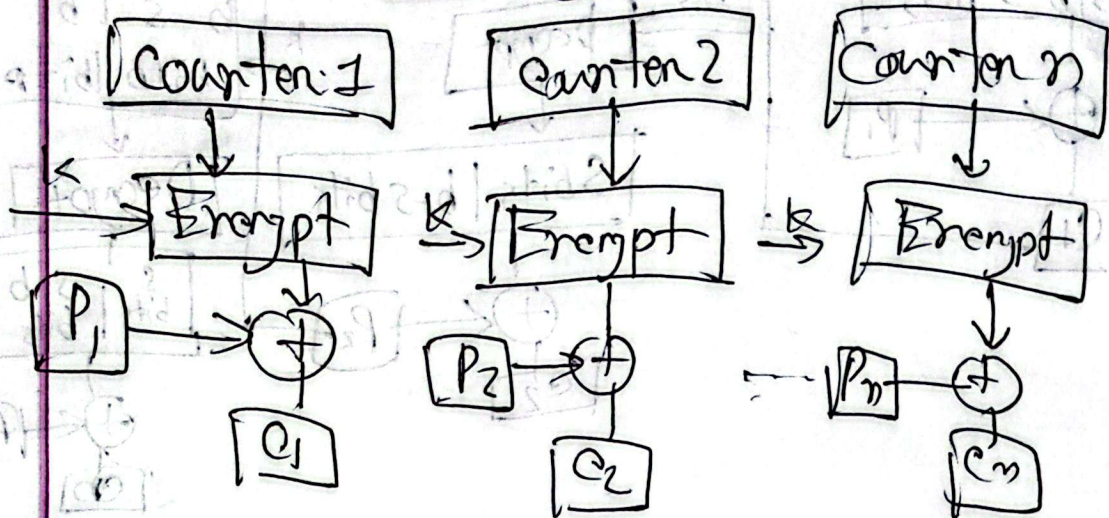
IT-21010

Decryption

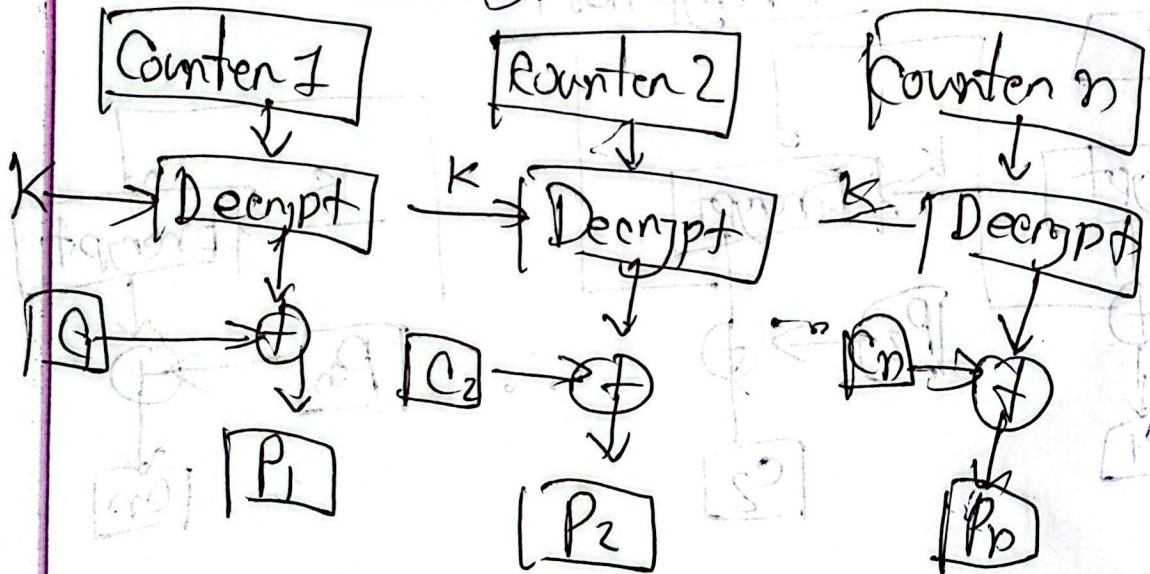
~~IV~~

Counter mode (CTR)

Encryption



Decryption



stop

Code:

```

Package org.example.rc5fx;
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;
import java.nio.ByteBuffer;
import java.nio.charset.StandardCharsets;
import java.util.Arrays;

```

```

public class RC5FX1 extends Application {
    private static final int W = 32;
    private static final int R = 12;
    private static final int B = 16;

```

```

public class RC5FX1 extends
    private static final int C = 4;
    private static final int P = 0x57E15163;
    private static final int Q = 0x9E3779B9;
    private static int rc4(int x, int y) {

```

return (acc) | (acc >> (w-j));

}

private static void negKeySetup

{ byte[] key, int[] S}

{ int[] L = new int[C];

for (int i = B-1; i >= 0; i--) {

L[i/4] = (L[i/4] << 8) + key[i],

}

S[0] = P;

for (int i = 1; i < 2 * (P+1); i++) {

S[i] = S[i-1] + Q;

int A = 0, B = 0;

int i = 0, j = 0;

for (int k = 0; k < 3 * Math.max(2 * (P+1), 2 * (Q+1)); k++)

A = S[i] = not1 (S[i] + A + B, 3);

B = L[j] = not1 (L[j] + A + B, (A+B));

21-2/06

01/05-11

$$i = (i+1) \% (2 * (R+1))$$

$$j = (j+1) \% C$$

private static void nesEncrypt(int[] s,
int[] data) {

int A = data[0]

int B = data[1]

A = A + s[0]

B = B + s[1]

for (int i = 1; i <= R; i++) {

A = not (A ^ B, B) + s[2 * i];

B = not (B ^ A, A) + s[2 * i + 1];

}

data[0] = A;

data[1] = B;

private static String encryptText(String plaintext)

byte[] key = new byte[B];

int[] s = new int[2 * (R+1)];

IT-2/0/0

01/5/20

nc5 keySetup (key, s);
byte[] plainBytes = PlainText.getBytes
(StandardCharsets.UTF_8);

int paddedLength = (plainBytes.length +

byte[] padded = Array.copyOf(plainBytes,
paddedLength);

String Builder cipherText = new String
Builder();

ByteBuffer buffer = ByteBuffer.wrap(padded);
while (buffer.hasRemaining()) {

int A = buffer.getShort();

int B = buffer.getShort();

int[] data = {A, B};

nc5 Encrypt (s, data);

cipherText.append (String.format("%08x",
0x00000000 + data[0] + data[1]));

return cipherText.toString().trim();

IT-21010

```
Scene Scene = new Scene (Layout, 500, 300);  
Stage.set Title ("AES Encryption Tool");  
Stage.show(); set Scene (Scene);  
Stage.show();  
public static void main (String [] args) {  
    launch (args)  
}
```

Output

Input: hello world

padding into 64 bit blocks.

Block 1: "Hello wo" (8 bytes)

Block 2: "rd10101010" (Padded)

Encrypted output:

07c9f2a2 b1557b56 2993583d
8f0268b3

original: Hello world.

skd

—